

Implement a logistic regression model to predict the likelihood of a disease using the Pima Indians Diabetes dataset. Compare the performance with and without feature scaling.

Aim

To implement **Logistic Regression using scikit-learn** on the **Pima Indians Diabetes dataset** and evaluate model performance **with and without feature scaling** using:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- ROC Curve and AUC

Algorithm

1. Load the dataset
2. Separate features and target
3. Split into training and testing sets
4. Train Logistic Regression model **without scaling**
5. Evaluate using metrics, confusion matrix, and ROC curve
6. Apply **feature scaling**
7. Train model again
8. Evaluate and compare results

Program

```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix, ConfusionMatrixDisplay,
```

```

roc_curve, roc_auc_score)

from sklearn.preprocessing import StandardScaler

# Load dataset

data = pd.read_csv("diabetes.csv")

# Features and target

X = data.iloc[:, :-1]

y = data.iloc[:, -1]

# Train-test split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# -----
# WITHOUT FEATURE SCALING
# -----

model_no_scale = LogisticRegression(max_iter=1000)

model_no_scale.fit(X_train, y_train)

y_pred_no_scale = model_no_scale.predict(X_test)

y_prob_no_scale = model_no_scale.predict_proba(X_test)[:, 1]

print("Without Scaling:")

print("Accuracy:", accuracy_score(y_test, y_pred_no_scale))

print("Precision:", precision_score(y_test, y_pred_no_scale))

print("Recall:", recall_score(y_test, y_pred_no_scale))

print("F1 Score:", f1_score(y_test, y_pred_no_scale))

# Confusion Matrix

```

```
ConfusionMatrixDisplay.from_predictions(y_test, y_pred_no_scale)

plt.title("Confusion Matrix (Without Scaling)")

plt.show()
```

```
# ROC Curve
```

```
fpr_ns, tpr_ns, _ = roc_curve(y_test, y_prob_no_scale)

auc_ns = roc_auc_score(y_test, y_prob_no_scale)
```

```
plt.figure()

plt.plot(fpr_ns, tpr_ns, label=f"AUC = {auc_ns:.2f}")

plt.plot([0,1], [0,1], linestyle='--')

plt.xlabel("False Positive Rate")

plt.ylabel("True Positive Rate")

plt.title("ROC Curve (Without Scaling)")

plt.legend()

plt.show()
```

```
# -----
```

```
# WITH FEATURE SCALING
```

```
# -----
```

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)

model_scaled = LogisticRegression(max_iter=1000)

model_scaled.fit(X_train_scaled, y_train)

y_pred_scaled = model_scaled.predict(X_test_scaled)
```

```
y_prob_scaled = model_scaled.predict_proba(X_test_scaled)[:, 1]
```

```
print("\nWith Scaling:")
```

```
print("Accuracy:", accuracy_score(y_test, y_pred_scaled))
```

```
print("Precision:", precision_score(y_test, y_pred_scaled))
```

```
print("Recall:", recall_score(y_test, y_pred_scaled))
```

```
print("F1 Score:", f1_score(y_test, y_pred_scaled))
```

```
# Confusion Matrix
```

```
ConfusionMatrixDisplay.from_predictions(y_test, y_pred_scaled)
```

```
plt.title("Confusion Matrix (With Scaling)")
```

```
plt.show()
```

```
# ROC Curve
```

```
fpr_s, tpr_s, _ = roc_curve(y_test, y_prob_scaled)
```

```
auc_s = roc_auc_score(y_test, y_prob_scaled)
```

```
plt.figure()
```

```
plt.plot(fpr_s, tpr_s, label=f"AUC = {auc_s:.2f}")
```

```
plt.plot([0, 1], [0, 1], linestyle='--')
```

```
plt.xlabel("False Positive Rate")
```

```
plt.ylabel("True Positive Rate")
```

```
plt.title("ROC Curve (With Scaling)")
```

```
plt.legend()
```

```
plt.show()
```